

B11 Series B

Multi-sample integration: batch effects and Harmony

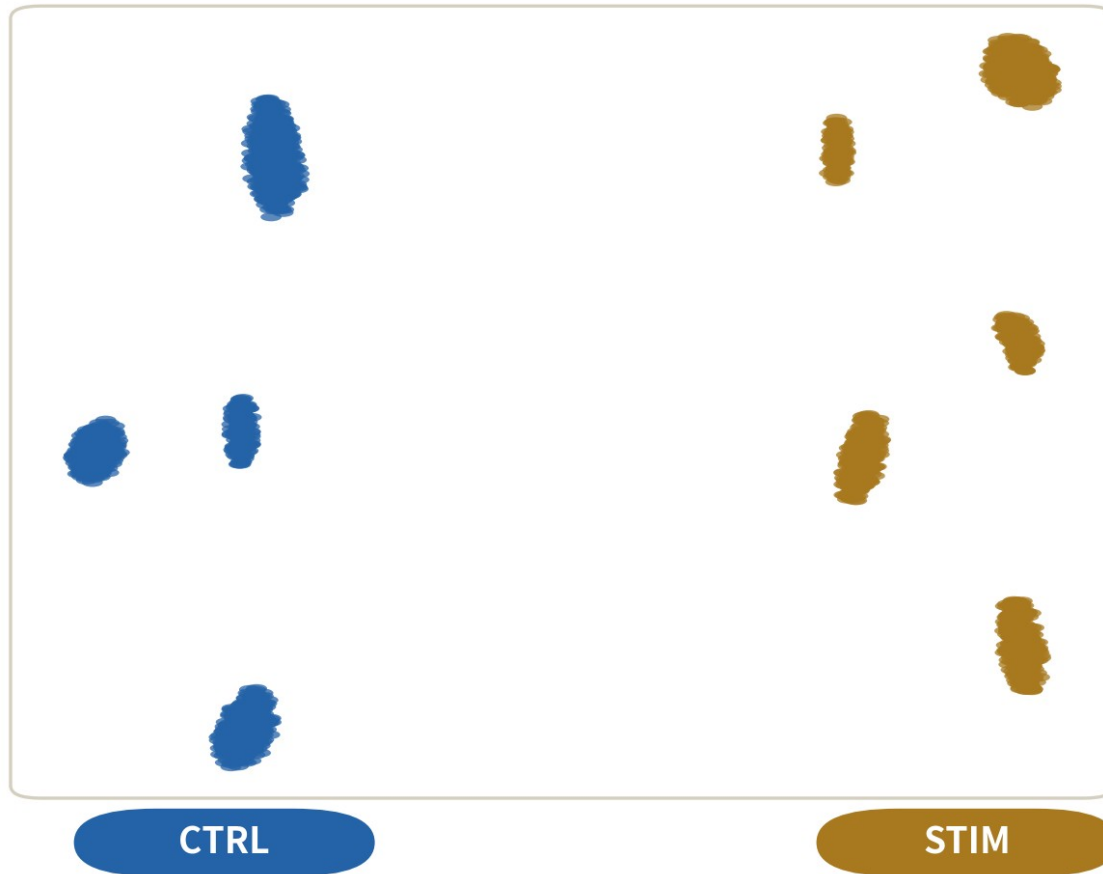
Run `IntegrateLayers` and `Harmony` in Seurat v5 — and tell when integration has gone too far.

About 40 min | Prerequisites: B5–B10 | Data: ifnb (control vs IFN-beta)

Two conditions, two islands

simulated data

merged data, no integration



Reading 1: batch effect

technical noise split identical cells in two —
integration should remove it

Reading 2: real biology

IFN-beta genuinely rewired every cell —
the separation is the signal

one picture, two readings — both true

Batch effect to remove, or stimulation response to keep? Both readings hold

WHERE THIS EPISODE STARTS

**The plot will not tell you which differences
are technical and which are biology.**

Anyone can run an integration tool. The hard part is knowing what it changed, what it bet on,
and when it lost.

What you will learn

- 1 Run CCA, RPCA and Harmony integration on ifnb under the v5 layer architecture
- 2 Explain why batch and biology look identical on a plot, and what integration bets on
- 3 Judge integration quality with known markers and mixing checks — and catch over-integration

01

What a batch effect actually is

Sort out the two kinds of variation before fixing anything

Technical vs biological variation

technical variation (nuisance)

- run date / operator
- reagent lot, dissociation
- 10x chemistry (v2 / v3)
- sequencing depth / machine

biological variation (the point)

- stimulated vs control (this episode)
- patient / individual
- tissue site, disease state
- sampling time point



both look identical on a UMAP — the plot cannot tell them apart; experimental design can, algorithms cannot

You want to discard the left and keep the right — but they arrive tangled together

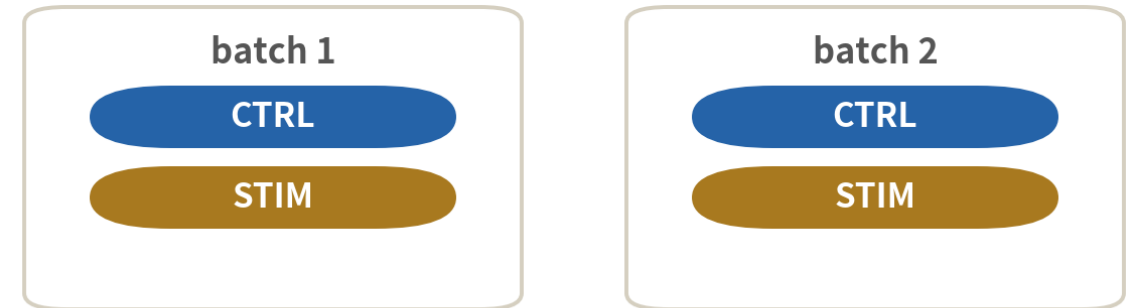
Confounded means unsolvable

confounded: batch = condition



**technical and biological effects
overlap 100% — nothing can untangle them**

balanced: both conditions in every batch



**batch effects become estimable
and the condition effect survives**

this decision happens before sequencing — once confounded, nothing in B11 can save you

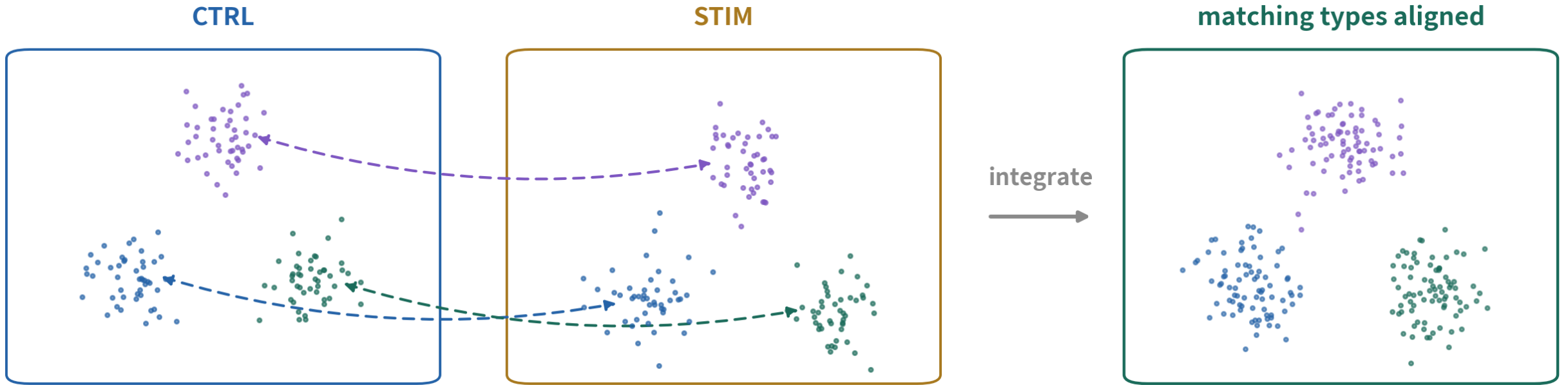
When batch equals condition, no algorithm can untangle them — this is an experimental-design problem

CORE IDEA

**Integration bets on one thing:
both batches share the same cell-type
structure.**

Win the bet and matching types align; lose it and real biology gets erased.

What integration does: pull matching types together



integration's bet: both batches share the same cell-type structure, so matching types can be pulled together

if the bet is wrong (a type exists in only one batch) it gets forced onto someone else — that is over-integration

Alignment happens at the level of cell types, not all cells — this distinction unlocks the episode

Why we leave pbmc3k behind

pbmc3k: one sample



one sample, one batch
— nothing to integrate

ifnb: two conditions



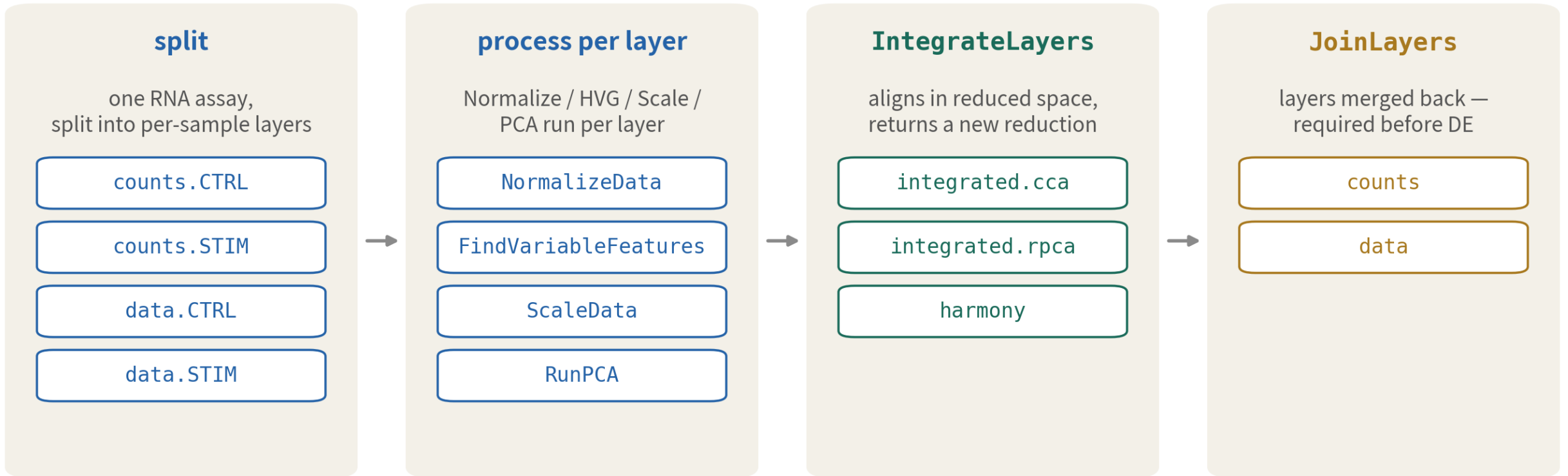
control vs IFN-beta
PBMC, 13,999 cells
(Kang et al. 2018)

with a built-in exam



the condition effect is
real biology — align
types without erasing
it

The Seurat v5 layer mental model



the correction lives in the reduction, not in expression values — JoinLayers cannot lose it

split → process per layer → IntegrateLayers → JoinLayers: the map for the whole pipeline

02

Load ifnb, split into layers

One line to download with SeuratData, one line to split

Load the ifnb dataset

```
library(Seurat)
library(SeuratData)
set.seed(1234) # 全系列固定，結果才能重現

# 第一次使用要先安裝資料集 (約 100 MB，之後有快取)
# installData("ifnb")
ifnb <- LoadData("ifnb")
ifnb <- UpdateSeuratObject(ifnb) # 把舊版物件升級到 v5 結構
ifnb
```

An object of class Seurat
14053 features across 13999 samples within 1 assay
Active assay: RNA (14053 features, 0 variable features)
1 layer present: counts

13,999 cells = 6,548 control + 7,451 stimulated; the condition label lives in ifnb\$stim

Split layers by condition

```
table(ifnb$stim)          # 條件標籤: CTRL / STM  
  
ifnb[["RNA"]] <- split(ifnb[["RNA"]], f = ifnb$stim)  
ifnb
```

CTRL STM
6548 7451

An object of class Seurat
14053 features across 13999 samples within 1 assay
4 layers present: counts.CTRL, counts.STM,
data.CTRL, data.STM

No cell moved — the same assay's data just went into separate drawers

Preprocessing runs per layer

```
# split 狀態下，這四步會對每個 layer 「各自」執行
ifnb <- NormalizeData(ifnb)
ifnb <- FindVariableFeatures(ifnb)
ifnb <- ScaleData(ifnb)
ifnb <- RunPCA(ifnb)
```

```
Normalizing layer: counts.CTRL  
Normalizing layer: counts.STM  
Finding variable features for layer counts.CTRL  
Finding variable features for layer counts.STM  
...
```

Messages print per layer — visible proof that each dataset is processed on its own

What this section did

One object



both conditions live in
one Seurat object, in
separate layers

Per-layer preprocessing



HVG, scaling and PCA
are computed per
condition

Nothing integrated yet



next: see what
happens without
integration — the step
most people skip

03

The unintegrated baseline

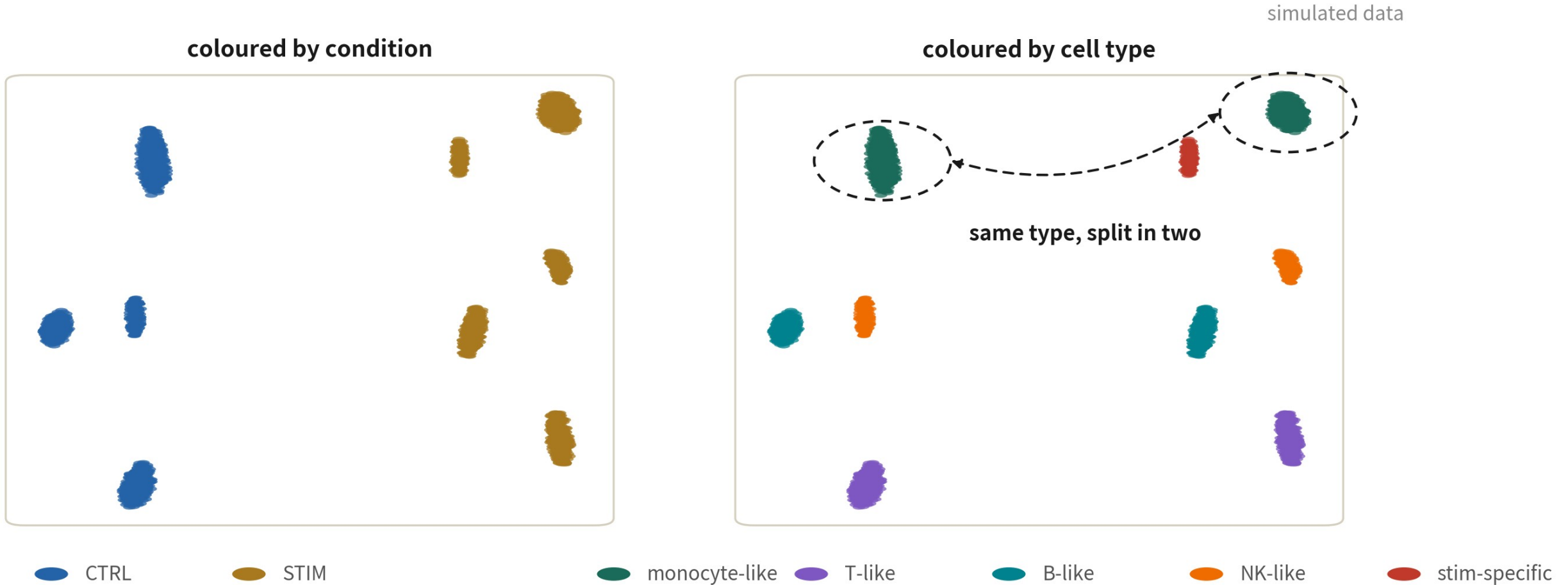
See the problem first, or you cannot judge the fix

Cluster and embed with no integration

```
ifnb <- FindNeighbors(ifnb, dims = 1:30, reduction = "pca")
ifnb <- FindClusters(ifnb, resolution = 0.5,
                     cluster.name = "unintegrated_clusters")
ifnb <- RunUMAP(ifnb, dims = 1:30, reduction = "pca",
               reduction.name = "umap.unintegrated")
DimPlot(ifnb, reduction = "umap.unintegrated",
        group.by = c("stim", "unintegrated_clusters"))
```

Name the reduction and the clusters — you will compare them with the integrated versions later

Unintegrated: matching types split in two



Left: two territories by condition. Right: each cell type shows up once on each side

IRON RULE

**Without the unintegrated picture,
you cannot tell what integration changed.**

This plot is your baseline. Keep it in the object — we will keep coming back to it.

04

IntegrateLayers: one interface, three methods

CCA, RPCA, Harmony — switching methods changes one argument

First: CCAIntegration

```
ifnb <- IntegrateLayers(  
  object      = ifnb,  
  method      = CCAIntegration,    # 換方法只換這裡  
  orig.reduction = "pca",          # 吃哪個未整合的降維  
  new.reduction = "integrated.cca", # 校正結果的名字  
  verbose      = FALSE  
)  
Reductions(ifnb)
```

```
[1] "pca"          "um ap .unintegrated "  
[3] "integrated.cca"
```

Look at the output: one new reduction. Not a single expression value changed

Choosing among the three

RPCAIntegration

anchors from reciprocal PCA projections

- conservative, rarely over-corrects
- fast; same-platform samples
- may under-correct big gaps

HarmonyIntegration

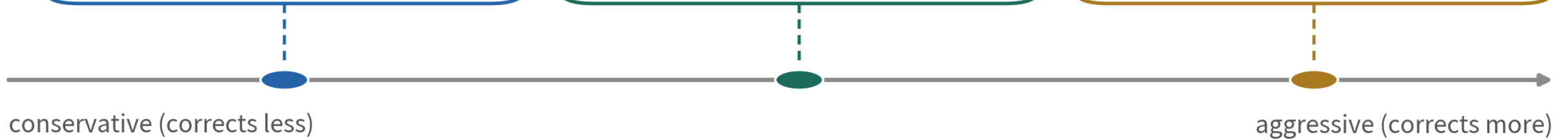
iterates in PC space: soft cluster, correct

- fast, light, scales well
- outputs a corrected embedding
- no corrected expression values

CCAIntegration

anchors via shared variation (CCA)

- strong: aligns big differences
- slow and memory-hungry
- most prone to over-correction



A spectrum of correction strength: RPCA conservative, Harmony fast in the middle, CCA strong but risky

CCA vs RPCA: two sides of one trade

Same anchor framework — the anchors are found in different spaces

CCA

shared variation directions

- aligns even under big condition shifts (our case)
- slow and memory-heavy
- the price: most likely to over-correct

RPCA

reciprocal PCA projection

- conservative: rarely erases biology
- fast; good for many same-platform samples
- may under-correct when batches differ a lot

Harmony and RPCA: one call each

```
# install.packages("harmony") # Harmony 後端要另外裝
ifnb <- IntegrateLayers(ifnb, method = HarmonyIntegration,
                        orig.reduction = "pca",
                        new.reduction = "harmony")
ifnb <- IntegrateLayers(ifnb, method = RPCAIntegration,
                        orig.reduction = "pca",
                        new.reduction = "integrated.rpca")
Reductions(ifnb)
```

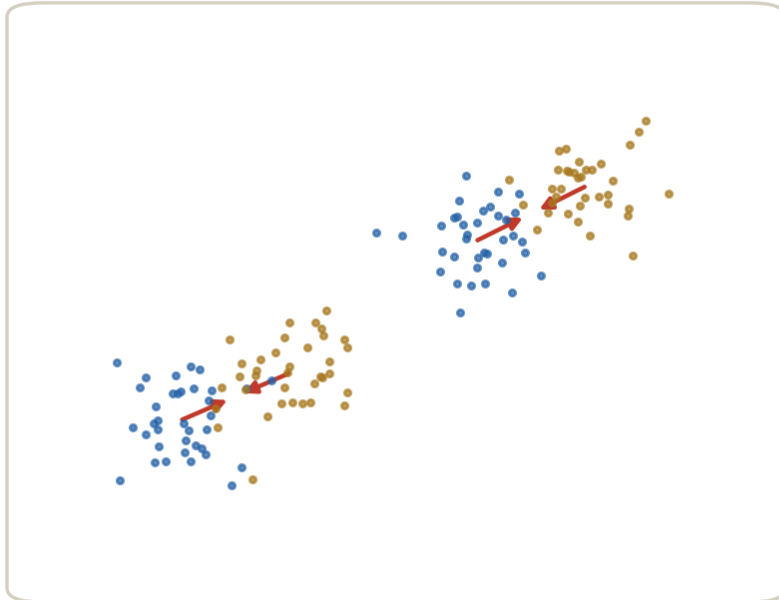
```
[1] "pca"          "um ap.unintegrated"
[3] "integrated.cca" "harmony"
[5] "integrated.rpca"
```

Three corrected versions coexist in one object; downstream you choose which to use

What Harmony actually does

simulated data

before: batches offset within clusters



1. soft clustering across batches



2. per-cluster, per-batch offsets

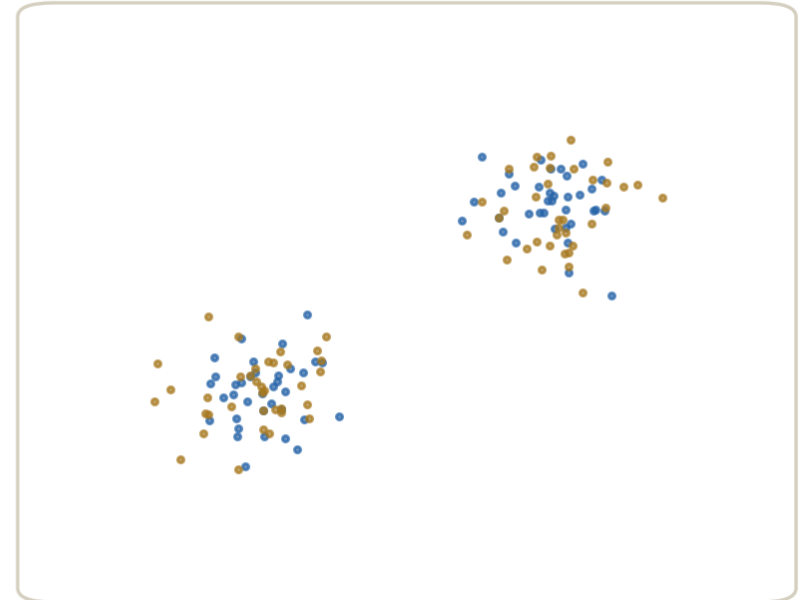


3. pull each batch toward the centre



4. repeat until mixed / converged

after: batches mixed within clusters



all of it happens in PC space (low-dimensional, fast); the output is a new reduction named harmony

Iterated in PC space: cluster, then pull each batch toward each cluster's centre — fast and light

Downstream after integration: point at the reduction

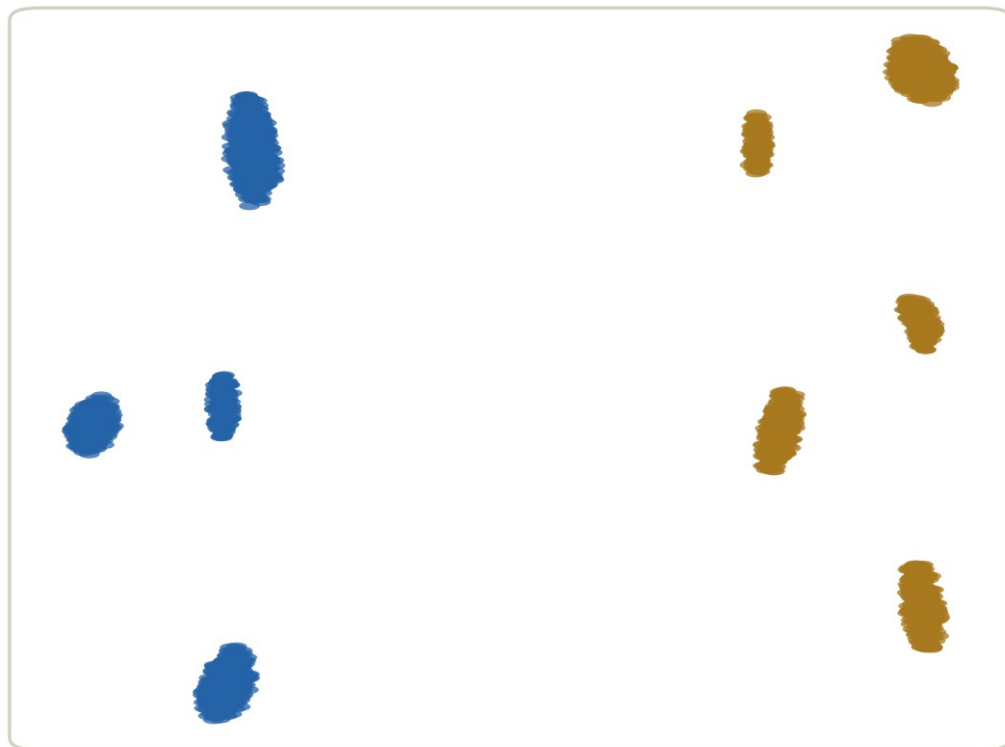
```
ifnb <- FindNeighbors(ifnb, reduction = "ham ony",  
                      dim s = 1:30)  
ifnb <- FindClusters(ifnb, resolution = 0.5,  
                     cluster.name = "ham ony_clusters")  
ifnb <- RunUMAP(ifnb, reduction = "ham ony", dim s = 1:30,  
                reduction.name = "um ap.ham ony")  
DimPlot(ifnb, reduction = "um ap.ham ony",  
         group.by = c("stim ", "ham ony_clusters"))
```

Exactly the B8-B9 moves — the only change is reduction = "harmony" instead of "pca"

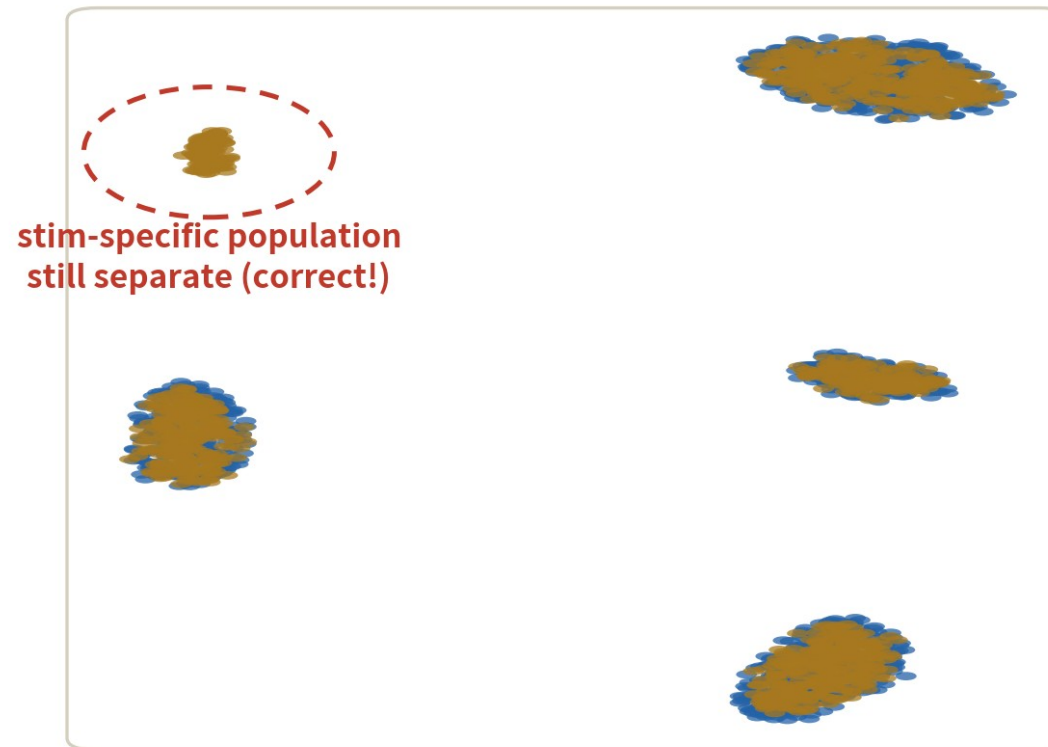
Before and after

simulated data

before: split by condition



after (Harmony): conditions mixed



CTRL

STIM

Matching types now overlap across conditions; the stim-specific population stays separate — that is success

JoinLayers: merge the drawers back

```
ifnb <- JoinLayers(ifnb)  
ifnb
```

An object of class Seurat
14053 features across 13999 samples within 1 assay
Active assay: RNA (14053 features, 2000 variable features)
2 layers present: data, counts

When: after integration and clustering, before any DE or marker testing

What this section did

Three integrations



CCA, RPCA and Harmony each produced a new reduction

Downstream re-pointed



clustering and UMAP now read harmony; expression never changed

Layers rejoined



JoinLayers merged counts and data; the correction lives safely in the reduction

05

Integration ran — did it work?

Three checks: look, count, and ask the markers

Checks 1 and 2: split views and numbers

1. 分開畫：同一群應該出現在兩邊的同一個位置

```
DimPlot(ifnb, reduction = "umap.harm ony", split.by = "stim ")
```

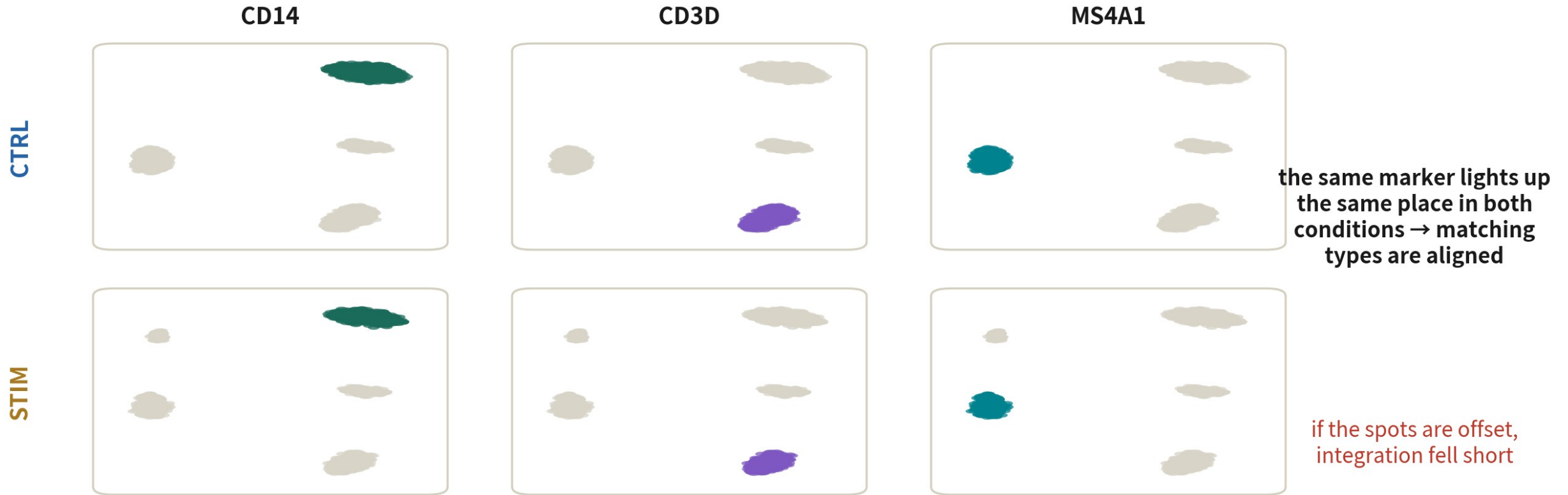
2. 每群的條件組成：大群應該接近對半

```
prop.table(table(ifnb$ham_ony_clusters, ifnb$stim ),  
            margin = 1) |> round(2)
```

	CTRL	STM
0	0.47	0.53
1	0.52	0.48
2	0.49	0.51
3	0.55	0.45
4	0.31	0.69

A heavily skewed cluster is not proof of failure — check its markers before judging

Check 3: known markers align across conditions



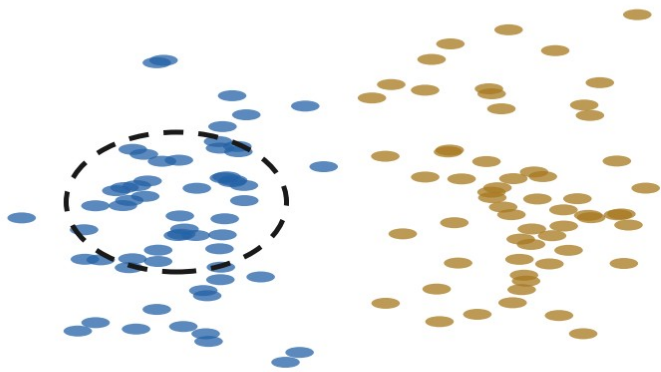
simulated data

CD14, CD3D, MS4A1 — each marker lights the same spot in both conditions

Quantifying mixing: the LISI and kBET intuition

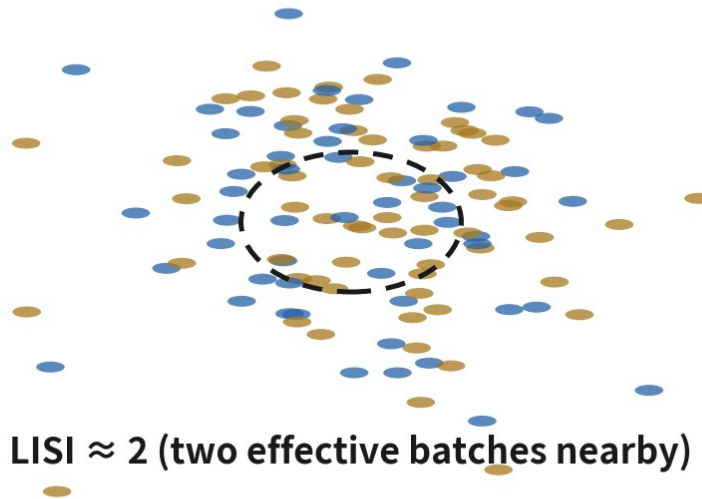
simulated data

unmixed: neighbours from one batch



$\text{LISI} \approx 1$ (one effective batch nearby)

well mixed: neighbours half and half



$\text{LISI} \approx 2$ (two effective batches nearby)

two common metrics

LISI

effective number of batches
in each neighbourhood

kBET

tests whether local batch mix
matches the global mix

careful: mixing should be judged within a cell type — perfect global mixing can mean over-correction

Concepts only for now — the key point is that mixing must be judged within a cell type

THE GOLDEN SENTENCE

**Integration aligns matching cell types —
it does not erase condition differences.**

Stimulated and control are supposed to differ. Perfect mixing everywhere is not success — it is an alarm.

What this section did

Three checks



split plots, per-cluster
condition shares,
known marker
positions

One canary



if the expected
condition-specific
population survives,
you have not over-
corrected

Two terms



LISI and kBET:
quantitative mixing
metrics, pocketed for
later

06

Where people get hurt

Three traps, each with its consequences laid bare

Trap 1: skipping the unintegrated baseline

What it is

Integrate first thing, without ever looking at the unintegrated picture.



Why it happens

Tutorials jump straight to integration, so it feels like step one of the standard pipeline.

What it costs you

Integration always hands you a nicely mixed plot — but you cannot say what it fixed, whether it was right, or how bad the problem originally was.

One-line insurance: keep the baseline in the object

```
# 整合前，先把未整合的 UMAP 以獨立名字存好（本集已做）  
ifnb <- RunUMAP(ifnb, dims = 1:30, reduction = "pca",  
               reduction.name = "umap.unintegrated")
```

```
# 之後任何時候都能一秒對照
```

```
DimPlot(ifnb, reduction = "umap.unintegrated",  
        group.by = "stim") |  
DimPlot(ifnb, reduction = "umap.harmoney",  
        group.by = "stim")
```

Reductions are cheap; with good names the before-after comparison is always one line away

Trap 2: correcting biology away as batch

What it is

Treating any separation as something to integrate away — including real differences like stimulated vs control.



Why it happens

The 'separated = batch effect' reflex is seductive, and strong methods like CCA really can flatten it.

What it costs you

The stimulation-specific population gets shoved into a control cluster, and 'what did the stimulus do' becomes unanswerable — your research question was deleted by preprocessing.

The canary test

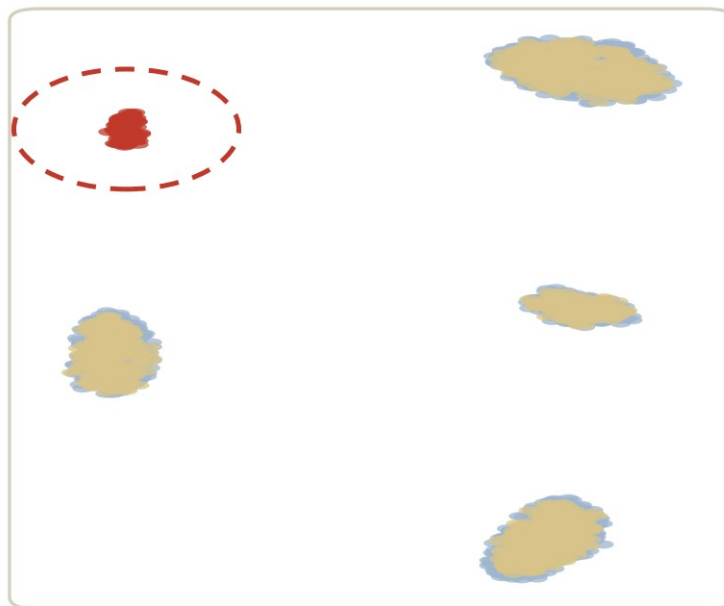
a condition-specific population you know should exist is your canary — if it vanishes, you over-corrected

simulated data

unintegrated



well integrated



over-integrated



canary alive: its own cluster

canary shoved into another cluster

CTRL

STIM

stim-specific population (canary)

A condition-specific population you know should exist is your canary — its disappearance is proof of over-correction

Trap 3: running DE on corrected values

What it is

After integrating, running FindMarkers for condition contrasts on the integrated assay or corrected output.



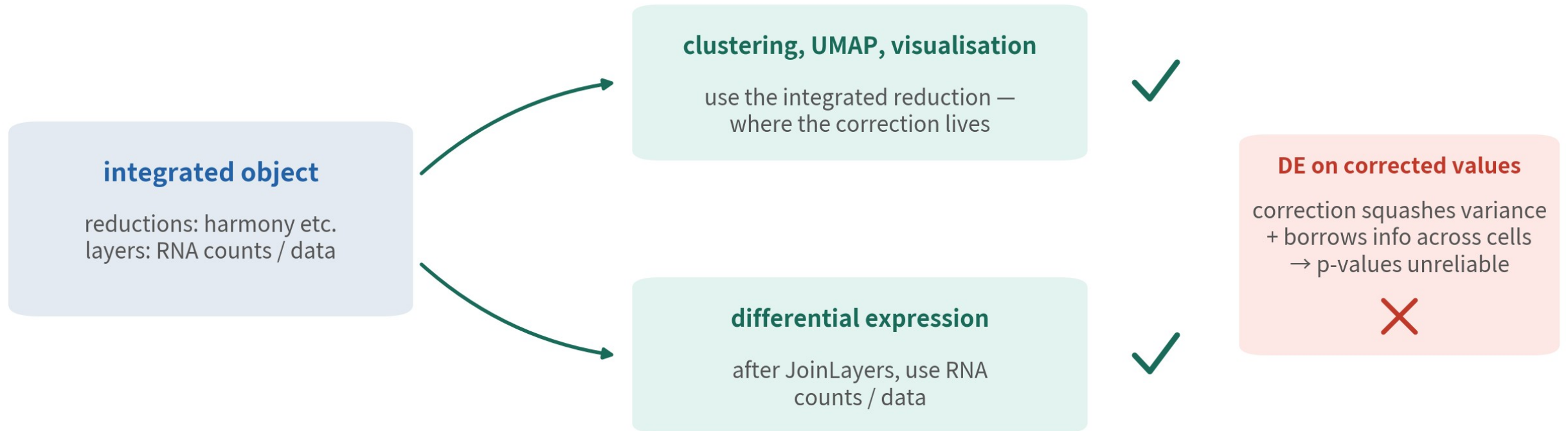
Why it happens

The v4-era IntegrateData produced an integrated assay that looks just like expression, so people reach for it.

What it costs you

Correction squashes variance and borrows information across cells — the test's assumptions are gone and p-values cannot be trusted. DE always goes back to RNA expression.

Geometry from integration, statistics from RNA



rule: use integration for geometry (distances, clusters, plots); do statistics on raw expression (B12 continues)

v5's `IntegrateLayers` returns only a reduction, blocking half the trap — the other half is forgetting `JoinLayers`

TRAPS, SUMMED UP

Integration is borrowed alignment, not new measurement.

Use it for layout — distances, clusters, plots — never for statistics. The measurements stay in the RNA layers.

5+

Strategy Room: Complex Samples

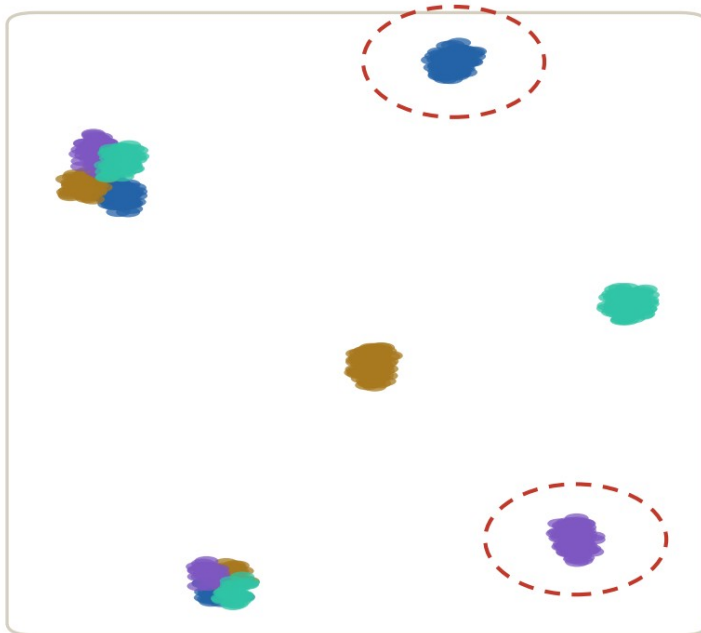
The standard route is done — now imagine the layers are not CTRL and STIM, but tumours from four patients

The integration dilemma: align the TME, never force the malignant cells

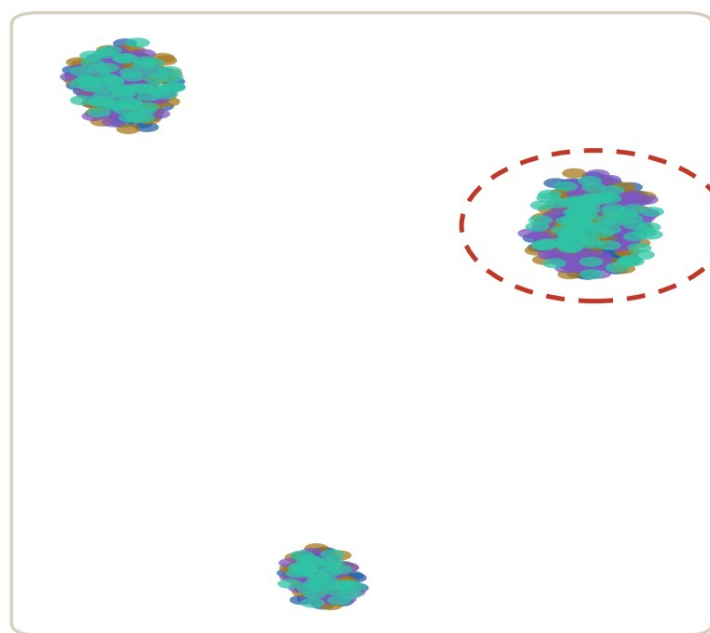
use the integrated space only to align and annotate normal cells; malignant analysis returns to the unintegrated space, per patient

simulated data

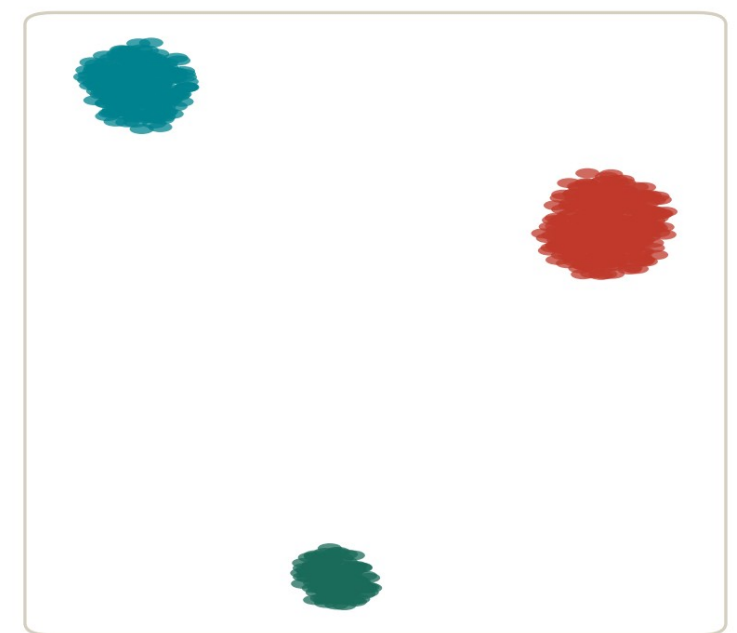
unintegrated, by patient



integrated, by patient



integrated, by cell class



malignant: one island per patient

TME: already mixed across patients

● patient 1 ● patient 2 ● patient 3 ● patient 4

malignant pulled together = biology erased

the plot looks clean — the price is already paid

● malignant ● immune ● oligodendrocyte

Immune and stroma are the same thing across patients — align them; malignant cells carry patient-specific CNV — pulling them together erases biology

The strategy table: what to integrate, what not to

The way out is not a smarter method — it is restricting what the integrated space is for

Cells	Which space	Why
Immune cells	integrated space: align and annotate	the same thing across patients — differences are mostly technical
Stroma, vessels, oligodendrocytes	integrated space: same logic	normal-cell subtype annotation happens here too
Malignant cells	unintegrated space: per-patient analysis	different things across patients — the difference is CNV, not batch
Comparing malignant across patients	compare state scores / meta-programs, not clusters	the standard move of the Tirosh / Neftel / Puram tumour atlases

Different tumours, different integration playbooks

GBM: the table as written



one patient one island,
clear CNV — TME in
the integrated space,
malignant per patient

Blood cancers: malignant looks normal



AML blasts resemble
normal progenitors
and CNV is often clean
— the island signal
vanishes; lean on a
normal-marrow
reference and the
differentiation block

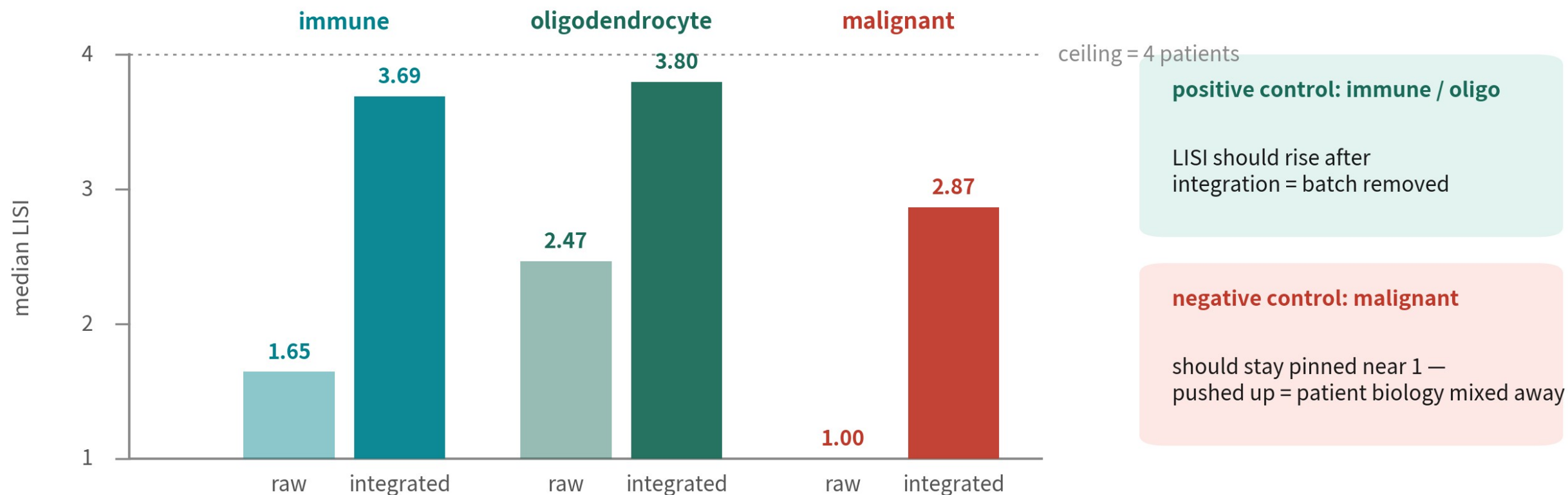
Multi-region: integrate within a patient



one patient's core and
margin share a CNV
lineage — integrate
them; across patients,
still separate

Positive and negative controls, read with LISI

simulated data



the negative control rarely 'passes' — methods are compared by who loses least, not who wins

The positive control (immune) should rise, the negative control (malignant) should barely move — methods are compared by who loses least

Reference atlases and label transfer: the other road

Normal cells: project and inherit



Azimuth / MapQuery projects onto a ready atlas — easiest and most accurate for immune subtypes

Malignant cells: never project



no atlas contains this patient's tumour — GBM often maps to astrocyte or OPC; a hint, not an answer

Malignancy: decided by CNV



patient-specific clustering + CNV + lineage, triangulated — the whole of B16

Episode checklist



you checked the design: is batch confounded with condition?



an unintegrated UMAP is saved as the baseline



trade-offs stated: RPCA safe / Harmony fast / CCA strong



known markers align across both conditions



the canary lives: condition-specific populations survived



JoinLayers before DE; statistics run on RNA expression

All six boxes ticked before you move to B12

ONE-LINE SUMMARY

**Look at the unintegrated baseline, then integrate —
and let the canary sign off on the result.**

Matching types aligned, condition effects alive, DE back on RNA. Three clauses, whole episode.

Check yourself 1

On an unintegrated UMAP the two conditions form separate islands. What does that show?

- A Definitely a batch effect — integrate it away
- B Definitely biology — keep it
- C Both technical and biological effects look like this — the plot alone cannot tell
- D The UMAP parameters are wrong

Answer C. Separation only says 'a big difference exists'; the plot never reveals its source. The judgement comes from experimental design (is batch confounded with condition?) and domain knowledge, not from the picture.

Check yourself 2

Which statement about the three integration methods is correct?

- A CCA is the most conservative and almost never over-corrects
- B RPCA is the strongest, best for wildly different batches
- C Harmony outputs only a corrected embedding, no corrected expression values
- D All three modify the counts

Answer C. Harmony iterates in PC space and yields a new reduction. A and B are exactly backwards — CCA is strong but risky, RPCA conservative; and none of the three touches counts in v5.

Check yourself 3

After integration one cluster is 98% STIM cells with ISG15-type interferon markers on top. You should...

- A Switch to a stronger method until it mixes
- B Declare the integration failed and rerun
- C Keep it — condition-specific populations should stay separate; this is biology
- D Delete the cluster and move on

Answer C. An interferon response is exactly what stimulation should produce — this cluster is the canary alive and well. A is trap 2 in person: forcing it to mix corrects your research question away.

NEXT EPISODE

B12: Differential expression, done right

With both groups aligned, it is time for the real question: which genes did stimulation change?

A naive FindMarkers gives 3,000 hits with p-values near $1e-200$ — hold the celebration. Why is that a statistical mirage?